

CSE370 : Database Systems
Project Report
Project Title : BracuLink

Group No : 12 , CSE370 Lab Section : 15, Summer 2026		
ID	Name	Contribution
24321232	Kazi Yasin Hridoy	33.33%
24321234	Al Jayed Rup	33.33%
23410390	P.M. Mushfiquir Shium	33.33%

Table of Contents

Section No	Content	Page No
1	Introduction	3
2	Project Features	3
3	ER/EER Diagram	5
4	Schema Diagram	6
5	Normalization	7
6	Frontend Development	10
7	Backend Development	11
8	Source Code Repository	13
9	Conclusion	13
10	References	14

Introduction

BracuLink is a web-based platform built exclusively for BRAC University students. Its purpose is to help students stay connected with their peers and coordinate around each other's class schedules something that currently happens through scattered Messenger groups and repeated messages asking "are you free now?"

After signing up with their university email, a student records the course sections they are enrolled in this semester. From that alone the platform builds their complete weekly routine automatically, laying out every class in a timetable and attaching each theory section's mandatory lab without the student having to enter it separately. Alongside the routine, a live indicator shows whether the student is in class at that moment or free, calculated against the current time.

The connection features are built around a friend system with deliberate privacy control. A student sends a friend request, and only once it has been accepted can the two see each other's full weekly routine and live free-or-busy status. This makes it easy to find a time when several friends are simultaneously free, without anyone's schedule being publicly visible. Each student also controls their own contact details individually a Facebook profile link can be shared openly, while a phone number is visible only if that student chooses to make it public.

All course and section information is synchronised automatically from the university's public course data feed, so seat counts, faculty initials, class timings and lab schedules always reflect real registration data rather than manually entered values.

The entire back end is built with Java and Spring Boot using Spring JDBC. No object relational mapping framework is used anywhere in the project: every SQL statement, including the database schema itself, is written by hand, which keeps the data layer fully visible and under the group's direct control.

Project Features

The table below lists the features implemented by each group member. All features are fully implemented and demonstrable in the running application.

ID, Name	Features [3 per member]
24321232, Kazi Yasin Hridoy	Main Features: Authentication, Enrollment, Notification. Still working on some additional features and planning to expand the project further in the future.
24321234, Al Jayed Rup	Main Features: Live course sync, Course search, Section picker
23410390, P.M. Mushfiquir Shium	Main Features: Profile + privacy, Friend system, Friend-gated viewing

Note: verify this distribution matches the actual division of work in your group before submission.

Normalization

1. First Normal Form (1NF)

Yes, the schema is in 1NF because each table has uniquely identifiable rows and attributes are stored as single values. Therefore, no decomposition is required.

Note: class_schedules and lab_schedules are JSON fields. If strict classical 1NF is required, these could be decomposed into separate schedule tables.

2. Second Normal Form (2NF)

Yes, the schema is in 2NF. Most tables have a single-attribute primary key (id), so partial functional dependencies cannot occur. FRIENDSHIP has a composite key (requester_id, addressee_id), but its non-key attributes depend on the whole composite key, not on either part alone.

2. Third Normal Form (3NF)

COURSE_SECTION is not in 3NF because of some transitive dependencies like $\text{courseCode} \rightarrow \text{courseName}$, $\text{courseCode} \rightarrow \text{courseType}$, creating a transitive dependency from the primary key; Similarly for user table, $\text{id} \rightarrow \text{studentId} \rightarrow \text{bracuEmail} \rightarrow \text{rest attributes}$ which is also a transitive dependency.

Frontend Development

- index.html — signup, one-time-password verification and login.
- dashboard.html — the generated weekly routine table and the live free/busy indicator.
- courses.html — course search, section listing with faculty, timings, room and available seats, and enrollment.
- friends.html — friend requests, friend list, and privacy-gated viewing of a friend's routine.
- profile.html — profile editing with per-field privacy controls.

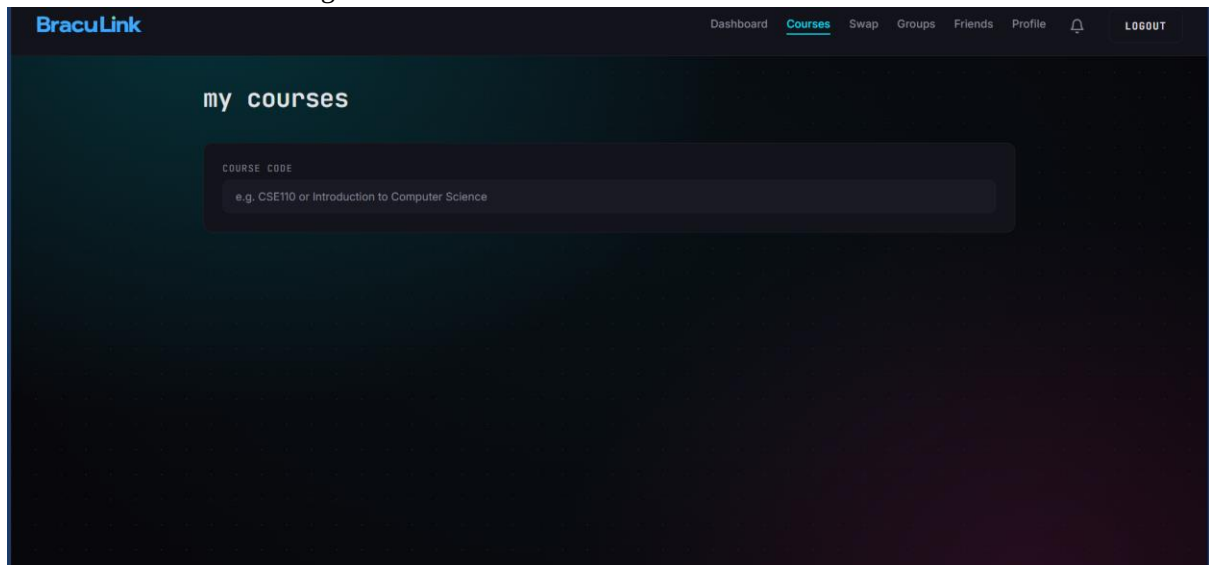
Contribution : 24321232, Kazi Yasin Hridoy

I made the login and signup page (index.html) and also worked on (friends.html). It has three parts: the signup form, the box for typing the OTP code, and the login form. I also wrote the shared js/api.js file, which keeps the login token and attaches it to every request the site sends, so the other pages do not have to handle that themselves. I also added the notification list that shows new messages to the user.

The screenshot displays the BracuLink web application interface. At the top, the 'BracuLink' logo is visible in blue and green. Below it, a message reads 'section swaps for bracu, done right'. The main focus is a dark-themed 'SIGN UP' form. The form includes the following fields: 'FULL NAME' with a placeholder 'your full name', 'STUDENT ID' with the placeholder 'hridoy@g.bracu.ac.bd', 'BRACU EMAIL' with the placeholder 'you@g.bracu.ac.bd', and 'PASSWORD' with a masked input '.....'. A bright blue 'CREATE ACCOUNT' button is positioned below the password field. At the bottom of the form, there is a 'LOG IN' section with a 'BRACU EMAIL' field containing the placeholder 'hridoy@g.bracu.ac.bd'.

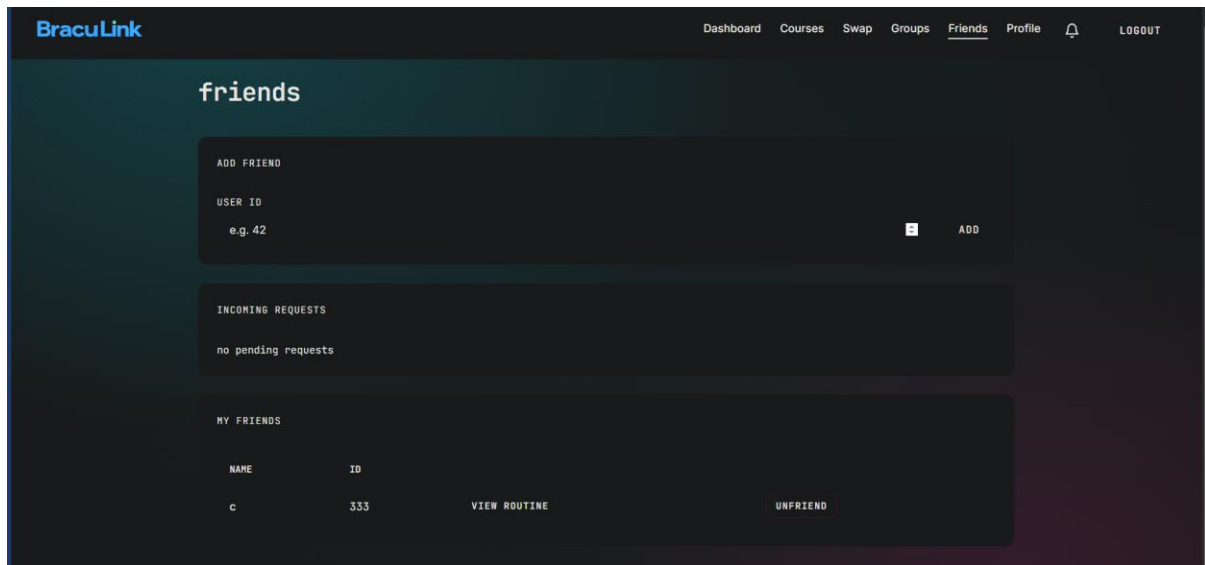
Contribution : 24321234, Al Jayed Rup

I made the courses page (courses.html) and the dashboard page (dashboard.html). On the courses page there is a search box for the course code, and after choosing a course it shows every section as a card with the faculty initial, class timing, lab timing, room and how many seats are left, along with the enroll button. On the dashboard I show the weekly routine as a table and a badge that says whether the student is free or in class right now.



Contribution of ID : 23410390, Name : P.M. Mushfiqur Shium

I made the friends page (friends.html) and the profile page (profile.html). The friends page shows incoming friend requests with accept and decline buttons, the list of current friends, and a button to view a friend's routine. The profile page lets a student edit their phone number and Facebook link, and turn the phone number public or private.



Backend Development

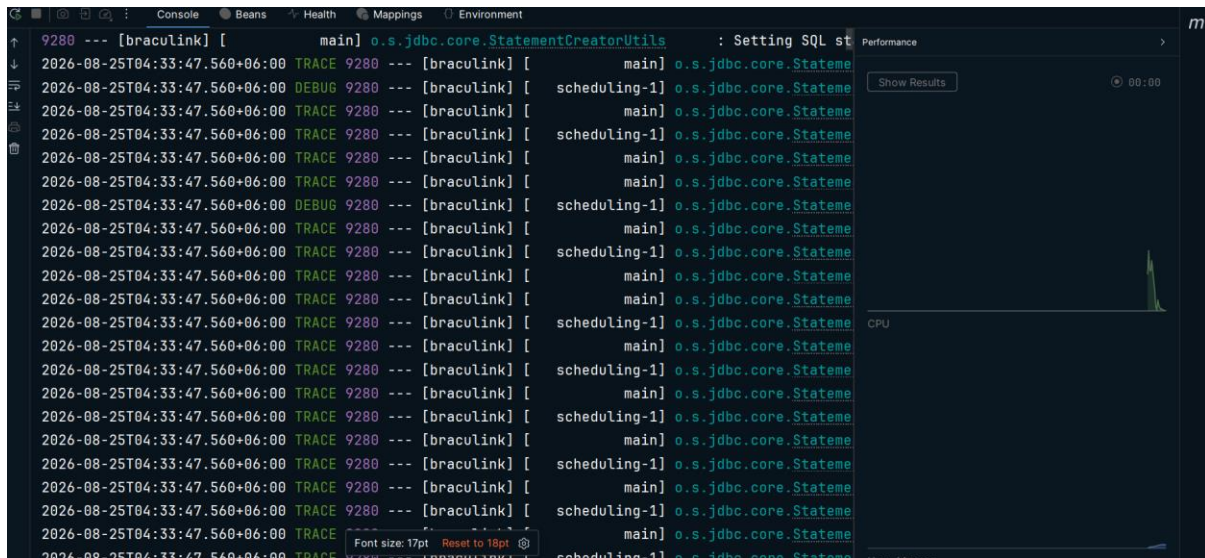
The back end is a Spring Boot application written in Java. Data access uses Spring JDBC through JdbcTemplate, and every SQL statement in the project, including the schema definition itself, is hand-written. No object relational mapping framework such as Hibernate or Spring Data JPA is used anywhere.

Architecture

Every feature follows the same three-layer path. A controller receives the HTTP request and converts JSON into a data transfer object. A service applies the business rules and makes all decisions. A data access object holds the SQL and communicates with MySQL. The result then travels back through the same layers and is returned as JSON. Keeping all SQL confined to the DAO layer means every query in the project can be located and reviewed in one place.

Notable implementation details

- Security: passwords are stored only as one-way BCrypt hashes. Login does not decrypt the stored value; it hashes the supplied password with the stored salt and compares the results. Signup is restricted to the @g.bracu.ac.bd domain and requires verification through a six-digit code that expires after ten minutes and is deleted once used.
- Stateless authentication: after login the server issues a signed JSON Web Token carrying the user identifier. A servlet filter validates this token on every request, so no server-side session state is kept.
- SQL injection prevention: every value is passed as a bound parameter placeholder. No user-supplied value is ever concatenated into a SQL string
- Idempotent data synchronisation: course data is fetched from the university feed on startup and every ten minutes thereafter. The upsert uses INSERT ... ON DUPLICATE KEY UPDATE keyed on the composite unique constraint (section_id, semester_session_id) and is executed as a batch, so repeated synchronisation updates existing rows instead of creating duplicates.



Contribution of ID : 24321232, Name : Kazi Yasin Hridoy

I worked on authentication, enrollment and notifications.

Authentication: AuthController receives the signup request and passes it to AuthService. The service checks that the email ends with @g.bracu.ac.bd, checks that the email and student ID are not already used, and hashes the password with BCrypt. Then UserDao runs the INSERT. OtpDao saves a six digit code with a ten minute expiry, and deletes it once it has been used. At login, the service compares the typed password with the stored hash and returns a JWT token.

Enrollment: EnrollmentController takes the course code and section name. EnrollmentService finds the real section id using CourseSectionDao, checks the student is not already enrolled in that course, and EnrollmentDao saves the row. Listing enrollments uses a single JOIN query, so it does not run one extra query for every row.

Notifications: NotificationService saves a notification through NotificationDao whenever something happens, such as a friend request. NotificationController lets the user see their notifications and mark them as read.

Contribution of ID : 24321234, Name: Al Jayed Rup

I worked on the course data features.

Live course sync: ConnectJsonClient downloads the university's public JSON feed. CourseSyncService turns that data into objects, and CourseSectionDao saves them using one INSERT ... ON DUPLICATE KEY UPDATE statement. Because of the unique key on section and semester, running the sync again updates the old rows instead of creating duplicates. It runs once when the application starts and then every ten minutes.

Course search: CourseController takes the text the user typed, CourseService passes it down, and CourseSectionDao runs a SELECT DISTINCT query with LIKE to find matching courses.

Section picker: the same controller and service are used, and CourseSectionDao returns each section with its faculty initial, class timing, lab timing, room and available seats. The seat count is calculated inside the SQL as capacity minus consumed seats, so MySQL does that work instead of Java.

Contribution of ID : 23410390, Name : P.M. Mushfiqur Shium

I worked on the profile and friend features.

Profile: UserController reads and updates the profile, UserService prepares the data, and UserDao runs the SELECT and UPDATE queries. The phone number is shown to other students only if the owner marked it public, and this check is done on the server, not in the browser.

Friend system: FriendshipController handles sending, accepting, declining and removing friends. FriendshipService makes sure a student cannot add themselves and that a friendship does not already exist in either direction, then FriendshipDao saves it. The friend list query checks both columns, so it finds the correct person no matter who sent the request first.

Friend-gated viewing: before showing anyone's routine, UserService calls areFriends. If it returns false the user gets a 403 error and the routine query never runs, so a student's schedule is only loaded when the two people are really friends.

Source Code Repository

The complete source code is publicly available at:

<https://github.com/KaziYasinHridoy/BracuLink>

The repository contains the Spring Boot back end, the hand-written schema file at src/main/resources/schema.sql, the static front end, and the automated test suite.

Conclusion:

Building BracuLink taught us how a real database application is made from beginning to end. We started with an EER diagram, turned it into a relational schema, and then wrote every table and every query by hand instead of letting a framework write them for us. Because of that, we always knew exactly what SQL our application was running.

The most useful thing we learned was where each kind of code belongs. The controller only handles the web request, the service holds the rules, and the DAO holds the SQL. Keeping these separate made it much easier for three people to work together, because each of us could build our own features without breaking someone else's part.

We also learned that the database can protect the data by itself. Foreign keys stop a row from pointing to something that does not exist, and unique keys stop the same row from being saved twice. These rules keep working even if there is a mistake in the Java code. Checking our schema against 1NF, 2NF and 3NF also showed us the places where our design is not perfectly normalized, and made us think about why we chose to keep it that way.

The project is working now, but there are still features we want to add, and we plan to keep improving it in the future.

References

Oracle Corporation, MySQL 8.0 Reference Manual. Available:
<https://dev.mysql.com/doc/refman/8.0/en/>

VMware, Spring Boot Reference Documentation. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

VMware, Spring Framework Documentation — Data Access with JDBC. Available:
<https://docs.spring.io/spring-framework/reference/data-access/jdbc.html>

BRAC University USIS course data feed. Available: <https://usis-cdn.eniamza.com/connect.json>